

TUTORIAL 01

Training a model

40 models, no experiment tracker, and five questions you will not be able to answer

DDM501 — AI in DevOps, DataOps, MLOps

1. Purposes

Models have been built with the same thing: a folder of scripts, a spreadsheet of results, and a filename convention. It works until someone asks a question about a model that was trained before, and then it does not work at all.

This tutorial walks you into that wall on purpose. You will train roughly 40 models with no tracking tool, and then be asked questions about them.

Tutorial 02 introduces MLflow. Every feature it shows maps to one of the questions.

2. Requirements

Python 3.11 or higher

The data ships inside scikit-learn package as a 120 KB file.

```
python3.11 -m venv .venv

# macOS / Linux
source .venv/bin/activate

# Windows PowerShell
.venv\Scripts\Activate.ps1

pip install -r requirements.txt
```

3. The data

The Wisconsin Diagnostic Breast Cancer set: 569 samples, 30 numeric features, 212 malignant and 357 benign. The features are measurements of cell nuclei — radius, texture, concavity and so on — computed from digitised images of fine needle aspirates.

3.1 Say what the model actually does

The model predicts the label a pathologist assigned. It does not diagnose cancer.

This is not pedantry. A model that reproduces historical labels at 99% has learned to agree with the people who wrote the labels.

3.2 The encoding

Value	Meaning
0	malignant — the case you must not miss
1	benign

3.3 Which mistake costs more

Accuracy is the wrong headline metric here. A model that calls every sample benign scores 63% and misses every cancer.

Mistake	This tutorial (diagnosis)	Lab 1 (credit scoring)
False negative	A malignant case is called benign. The patient goes home.	A defaulter is approved. The bank loses the loan.
False positive	A benign case is flagged. An unnecessary biopsy, and a frightened patient.	A good customer is declined. The bank loses the margin.
Which dominates	The false negative, by a wide margin. Optimise recall on the malignant class.	Roughly balanced, and the balance is a business decision, not a technical one.

4. Step 1 — one model, one number

```
python scripts/step1_baseline.py
```

A single random forest, a fixed split, and the metrics you would report in a meeting.

```
rows=569  features=30  malignant=212  benign=357

accuracy          0.9561
roc_auc           0.9932
recall(malignant) 0.9286

confusion matrix (rows = truth)
malignant:  caught 39   missed 3   <- these are the costly ones
benign:     flagged 2    cleared 70
```

Three malignant cases missed out of 42. Whether that is acceptable is not a question the script can answer.

Answer:

1. What was the test ROC AUC, to four decimals?
2. What random_state produced it?
3. How many trees were in the forest?
4. Which scikit-learn version pickled models/model.joblib?
5. Was the data scaled?

5. Step 2 — 30 models

Randomised search over three model families: random forest, support vector machine, and logistic regression. We have 30 configurations, five-fold cross-validation, ranked by ROC AUC.

```
python scripts/step2_random_search.py
```

```
python scripts/step2_random_search.py --seed 7
python scripts/step2_random_search.py --no-scaling
```

Run all three. Each takes about thirty seconds. Here is what they produced on the machine this tutorial was written on:

Run	Winning	Best mean ROC AUC
default	SVC	0.99486
--seed 7	LogisticRegression	0.99438
--no-scaling	RandomForestClassifier	0.99239

Read that table again. Changing only the random seed changed which model family won. Nothing about the data changed, nothing about the search space changed.

5.1 The five questions

Now answer these:

1. Which of the three runs produced the models/model.joblib you have on disk right now?
2. Which run had the best recall on malignant cases?
3. Run 2 tried thirty configurations. What were the other twenty-nine?
4. Reproduce the seventeenth configuration of run 2.
5. Two teammates run this at the same time on a shared machine. What happens to models/model.joblib?

cannot answer any of them

1. The file has one name and each run overwrote it. There is no record of which won.
2. The search optimised ROC AUC. Recall was never computed, let alone stored.
3. They existed in memory for thirty seconds and were discarded.
4. The seed is in your shell history, if you have not closed the terminal.
5. One silently overwrites the other, and neither of you finds out.

6. Step 3 — the logbook

The obvious fix is a CSV. One row per run, one column per thing you care about. Almost every team builds this before adopting a tracking tool, and it is a real improvement over nothing.

```
python scripts/step3_logbook.py --tag baseline
python scripts/step3_logbook.py --tag deep --max-depth 3
python scripts/step3_logbook.py --tag baseline --n-estimators 400
```

Three runs later, logs/results.csv holds:

```
timestamp,tag,n_estimators,max_depth,seed,roc_auc,recall_malignant,sklearn,python
2026-08-28T11:45:52,baseline,200,,42,0.99322,0.92857,1.6.0,3.11.15
```

```
2026-08-28T11:45:54,deep,200,3,42,0.99140,0.92857,1.6.0,3.11.15
2026-08-28T11:45:56,baseline,400,,42,0.99322,0.92857,1.6.0,3.11.15
```

First, rows 1 and 3 have identical ROC AUC with 200 and 400 trees. Doubling the forest changed nothing measurable.

Second, you ran `--tag baseline` twice, and there is exactly one `models/model_baseline.joblib` on disk. The second run overwrote the first. Two rows in the logbook, one file, and no way to tell which row the file belongs to.

6.1 What the CSV still cannot hold

1. You now want to try an SVM with a gamma. Where does that column go, and what do the older rows contain for it?
2. Which row produced `model_baseline.joblib`? What if you ran the same tag three times?
3. You want the confusion matrix and the ROC curve for each run. Where do they live?
4. Two people run this at once. Open the CSV and look at the last lines.
5. Six months from now, which commit of this repository produced row 3?

7. Step 4 — did the winner actually win?

Step 2 ranked 30 configurations and declared a best. A 5-fold cross-validation score is a single draw from a distribution, and you cannot see the width of a distribution from one draw. This step repeats the folds three times so every configuration gets a standard deviation as well as a mean.

```
python scripts/step4_noise_floor.py+
top 10 configurations      (roc_auc, RepeatedStratifiedKFold 5 x 3)

 1. 0.99212 +/- 0.00590 #####
 2. 0.99196 +/- 0.00616 #####
 3. 0.99184 +/- 0.00629 #####
 4. 0.99161 +/- 0.00637 #####
 5. 0.99159 +/- 0.00635 #####
 6. 0.99157 +/- 0.00633 #####
 7. 0.99151 +/- 0.00714 #####
 8. 0.99120 +/- 0.00687 #####
 9. 0.99118 +/- 0.00643 #####
10. 0.99083 +/- 0.00825 #####

gap, rank 1 to rank 10    0.00129
std deviation of rank 1   0.00590
```

8. What you lost

What you could not do	MLflow concept
Tell which run produced a file on disk	run id
Add a column for a new hyper-parameter without breaking old rows	parameters
Keep a confusion matrix or a plot with a run	artifacts
Compare forty runs without reading a CSV by eye	the tracking UI
Two people logging at once without corruption	tracking server
Say which code produced a result six months later	source version
Point production at one specific model and move it later	registry + alias
Know which sklearn version pickled a model	model flavour metadata